Guia Completa: Deploy Multi-Entorno (Produccion + Staging)

Sistema Blitz — CI/CD con GitHub Actions + Docker Compose + VPS IONOS

Tabla de Contenidos

1. Que vamos a lograr
2. Como funciona (la teoria simple)
3. Que archivos se cambiaron
4. PASO 1: Configurar GitHub Environments
5. PASO 2: Configurar Secrets en GitHub
6. PASO 3: Preparar el VPS
7. PASO 4: Primer deploy manual
8. PASO 5: Probar el flujo automatico
9. Como acceder a cada entorno
10. Diagrama del flujo completo
11. Troubleshooting
12. Referencia de comandos
13. Checklist final

---

1. Que vamos a lograr

Dos versiones de la app corriendo al mismo tiempo en **un solo servidor VPS**:

```
PRODUCCION (lo que ven los usuarios reales)
   URL:    https://jesuslab135.com
   Rama:   main
   Ruta:   /root/app/infra/        ← docker compose corre desde aqui

STAGING (donde pruebas antes de subir a produccion)
   URL:    http://jesuslab135.com:8080
   Rama:   development
   Ruta:   /root/app-dev/infra/    ← docker compose corre desde aqui
```

El flujo automatico:

```
Tu haces push a "development"
        |
        v
GitHub Actions detecta la rama
        |
```

```
        v
Construye imagen Docker → sube a GHCR con tag :dev
        |
        v
Se conecta al VPS por SSH
        |
        v
Descarga imagen nueva en /root/app-dev/infra/
        |
        v
Reinicia contenedores de staging
        |
        v
Listo! Prueba en http://jesuslab135.com:8080
```

Lo mismo pasa con `main`, pero despliega en `/root/app/infra/` con tag `:latest`.

---

2. Como funciona (la teoria simple)

El problema: dos apps que chocan

Si corres dos copias de la misma app con Docker, todo choca:

Recurso	Produccion quiere	Staging quiere	Resultado
Puerto del backend	8000	8000	ERROR: puerto ocupado
Nombre del contenedor	blitz_backend	blitz_backend	ERROR: nombre duplicado
Base de datos	postgres_data	postgres_data	DESASTRE: comparten datos

La solucion: COMPOSE_PROJECT_NAME

Docker Compose tiene una variable que controla TODO el namespacing. Cuando la defines, Docker le pone ese nombre como **prefijo** a cada recurso:

```
COMPOSE_PROJECT_NAME=blitz-prod        COMPOSE_PROJECT_NAME=blitz-dev
├── blitz-prod-backend-1               ├── blitz-dev-backend-1
├── blitz-prod-db-1                    ├── blitz-dev-db-1
├── blitz-prod_postgres_data           ├── blitz-dev_postgres_data  ← BD SEPARADA
└── blitz-prod_blitz-net               └── blitz-dev_blitz-net      ← RED SEPARADA
```

Un solo `compose.yaml` para ambos entornos. La diferencia la hace el `.env` de cada directorio.

Los puertos

Cada entorno usa puertos diferentes para no chocar:

```
Produccion:                     Staging:
  Nginx:    80 / 443 (HTTPS)     Nginx:    8080 (HTTP)
  Backend:  8000                  Backend:  8001
  PGAdmin:  5050                  PGAdmin:  6060
```

Las imagenes Docker

```
push a main        → ghcr.io/jesuslab135/blitz-backend:latest
push a development → ghcr.io/jesuslab135/blitz-backend:dev
```

Ruta interna del contenedor vs ruta del VPS

Esto es importante y puede confundir. Son dos cosas diferentes:

```
RUTA EN EL VPS (donde estan tus archivos en el servidor):
  /root/app/          ← Produccion
  /root/app-dev/      ← Staging

RUTA DENTRO DEL CONTENEDOR (donde vive el codigo de Django):
  /app/core/          ← Igual en AMBOS contenedores

No hay conflicto: son contenedores separados.
Es como dos departamentos en edificios diferentes.
Ambos pueden tener una "sala" sin que choquen.
```

Archivos secretos (Firebase JSON)

El archivo `squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json` esta en `.gitignore`, asi que:

- **NO esta en el repositorio de GitHub**
- **NO esta dentro de las imagenes Docker** que construye GitHub Actions
- Se coloca **manualmente** en el VPS
- Se conecta al contenedor via **volume mount** en el compose.yaml

```
VPS (host):                                   Contenedor Docker:
/root/app/backend/squapup-...json    →→→     /app/core/squapup-...json
                                     volume
                                     mount
```

---

3. Que archivos se cambiaron

Archivos modificados

Archivo	Que se hizo	Por que
`backend/Dockerfile`	`/app-dev` → `/app` en todas las rutas	Estandarizar ruta interna del contenedor para que coincida con la imagen de produccion actual
`backend/scripts/entrypoint.sh`	`cd /app-dev/core` → `cd /app/core`	Misma razon
`frontend/Dockerfile`	`/app-dev` → `/app` + entrypoint para refrescar volumen	Estandarizar ruta + fix: named volumes no se actualizan solos en deploys
`infra/compose.yaml`	Reescrito completo	Parametrizado para multi-entorno + volume mount de Firebase credentials
`backend/.github/workflows/deploy.yml`	Reescrito	Detecta rama, usa GitHub Environments
`frontend/.github/workflows/deploy.yml`	Reescrito	Mismo patron que backend

Archivos nuevos

Archivo	Que es
`infra/.env.production`	Template del `.env` para produccion
`infra/.env.staging`	Template del `.env` para staging
`infra/nginx/staging.conf`	Nginx HTTP-only para staging
`infra/GUIA-DEPLOY-MULTI-ENTORNO.md`	Esta guia

Bugs que se corrigieron

Bug	Donde	Que pasaba
Port mismatch	compose.yaml	Daphne escucha en 8000, pero el compose mapeaba 8001:8001
COPY path roto	frontend/Dockerfile	WORKDIR era /app-dev pero COPY buscaba en /app/dist
Firebase no cargaba	compose.yaml	El JSON de Firebase no tenia volume mount, Django no podia leerlo
Frontend no se actualizaba	frontend/Dockerfile	El named volume `frontend_dist` solo se llena la primera vez; deploys posteriores servian archivos viejos

---

4. PASO 1: Configurar GitHub Environments

> Hacer esto en **AMBOS repos** (backend y frontend).

4.1 Abrir Settings del repositorio

1. Ve a `https://github.com/TU-USUARIO/blitz-backend`
2. Clic en **Settings** (pestana arriba a la derecha)
3. En el menu izquierdo, busca **Environments**

4.2 Crear environment "production"

1. Clic en **"New environment"**
2. Nombre: `production` (exacto, minusculas)
3. Clic en **"Configure environment"**
4. (Opcional) Activa **"Required reviewers"** → agrega tu usuario
   - Asi cada deploy a produccion necesita tu aprobacion manual
5. Baja a **"Environment secrets"** → **"Add secret"**

```
Name:   VPS_DEPLOY_PATH
Value:  /root/app/infra
```

6. Guardar

4.3 Crear environment "staging"

1. Vuelve a Settings > Environments > **"New environment"**
2. Nombre: `staging`
3. **"Configure environment"**
4. NO actives "Required reviewers" (staging se deploya solo)
5. **"Environment secrets"** → **"Add secret"**

```
Name:   VPS_DEPLOY_PATH
Value:  /root/app-dev/infra
```

6. Guardar

4.4 Repetir en el repo de Frontend

Exactamente lo mismo:

- `production` → `VPS_DEPLOY_PATH` = `/root/app/infra`
- `staging` → `VPS_DEPLOY_PATH` = `/root/app-dev/infra`

---

5. PASO 2: Configurar Secrets en GitHub

> Estos secretos son **compartidos** (mismo VPS para ambos entornos). Hacer en **AMBOS repos**.

5.1 Ir a los secrets del repositorio

Settings > **Secrets and variables** > **Actions** > **"New repository secret"**

5.2 Agregar 3 secretos

VPS_HOST

```
Name:   VPS_HOST
Value:  85.215.xxx.xxx    ← IP publica de tu VPS IONOS
```

VPS_USER

```
Name:   VPS_USER
Value:  root
```

VPS_SSH_KEY

```
Name:   VPS_SSH_KEY
Value:  (contenido completo de tu clave privada SSH)
```

Como obtener la clave SSH (en tu computadora LOCAL):

```
# Si ya te conectas al VPS con SSH, tu clave esta aqui:
cat ~/.ssh/id_rsa
# O:
cat ~/.ssh/id_ed25519
```

Copia TODO, incluyendo `-----BEGIN` y `-----END`:

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUA...
...muchas lineas...
-----END OPENSSH PRIVATE KEY-----
```

Si NO tienes clave SSH:

```
# Generar clave nueva (en tu computadora LOCAL)
ssh-keygen -t ed25519 -C "github-actions-deploy"
# Presiona Enter para todo (dejar passphrase vacia)

# Copiar la clave PUBLICA al VPS
```

```
ssh-copy-id root@85.215.xxx.xxx

# Verificar (no debe pedir contrasena)
ssh root@85.215.xxx.xxx "echo OK"

# Copiar la clave PRIVADA para GitHub
cat ~/.ssh/id_ed25519
# ← Pega esto como VPS_SSH_KEY en GitHub
```

Importante

Si antes tenias un secret `VPS_DEPLOY_PATH` a nivel de **repositorio**, eliminalo. Ese secret ahora vive dentro de cada Environment.

---

6. PASO 3: Preparar el VPS

Conectate al VPS:

```
ssh root@85.215.xxx.xxx
```

6.1 Verificar Docker

```
docker --version
# Debe mostrar Docker version 24+ o superior

docker compose version
# Debe mostrar Docker Compose version v2+
```

6.2 Tu produccion actual (no tocar, solo verificar)

```
ls /root/app/
# Debe mostrar: 700  backend  frontend  infra

ls /root/app/infra/
# Debe mostrar: compose.yaml  .env  (y tal vez .env.save)

ls /root/app/backend/
# Debe mostrar: squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json
```

6.3 Crear la estructura de Staging

```
# Crear todos los directorios necesarios
mkdir -p /root/app-dev/backend
```

```
mkdir -p /root/app-dev/frontend
mkdir -p /root/app-dev/infra/nginx
```

6.4 Copiar el Firebase JSON a staging

```
cp /root/app/backend/squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json \
   /root/app-dev/backend/squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json
```

6.5 Subir el compose.yaml al VPS

Desde tu **computadora local** (carpeta `infra/`):

```
# Subir compose.yaml a staging
scp compose.yaml root@85.215.xxx.xxx:/root/app-dev/infra/compose.yaml

# IMPORTANTE: Tambien actualizar el de produccion (tiene el volume mount de
Firebase)
scp compose.yaml root@85.215.xxx.xxx:/root/app/infra/compose.yaml
```

6.6 Subir las configuraciones de Nginx

```
# Nginx para PRODUCCION (con SSL) — ya deberia existir, pero por si acaso:
scp nginx/default.conf root@85.215.xxx.xxx:/root/app/infra/nginx/default.conf

# Nginx para STAGING (sin SSL, HTTP simple)
scp nginx/staging.conf root@85.215.xxx.xxx:/root/app-dev/infra/nginx/default.conf
```

> **Atencion**: El archivo se llama `staging.conf` en tu repo, pero se copia como `default.conf` en el VPS. Esto es porque el compose.yaml siempre monta `./nginx/default.conf`.

6.7 Crear el .env de PRODUCCION

> **Si ya tienes un .env en /root/app/infra/ que funciona**, solo necesitas agregar las variables nuevas.

```
nano /root/app/infra/.env
```

Asegurate de que tenga TODAS estas variables (agrega las que falten):

```
# --- Docker Compose ---
COMPOSE_PROJECT_NAME=blitz-prod
COMPOSE_PROFILES=ssl
IMAGE_TAG=latest
```

```
# --- Puertos ---
HTTP_PORT=80
HTTPS_PORT=443
BACKEND_PORT=8000
PGADMIN_PORT=5050

# --- Base de Datos (tus valores reales) ---
DB_USER=tu_usuario_bd
DB_PASSWORD=tu_password_bd
DB_NAME=tu_nombre_bd

# --- Django ---
SECRET_KEY=tu-secret-key-de-produccion

# --- Firebase ---
FIREBASE_CREDENTIALS_PATH=squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json

# --- Stripe ---
STRIPE_SECRET_KEY=sk_live_xxx
STRIPE_PUBLISHABLE_KEY=pk_live_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx

# --- PGAdmin ---
PGADMIN_EMAIL=admin@blitz.com
PGADMIN_PASSWORD=password_seguro
```

Guardar: `Ctrl+O`, `Enter`, `Ctrl+X`

6.8 Crear el .env de STAGING

```
nano /root/app-dev/infra/.env
```

Pega esto y edita con valores de staging:

```
# --- Docker Compose ---
COMPOSE_PROJECT_NAME=blitz-dev
IMAGE_TAG=dev

# --- Puertos (DIFERENTES de produccion!) ---
HTTP_PORT=8080
HTTPS_PORT=8443
BACKEND_PORT=8001
PGADMIN_PORT=6060

# --- Base de Datos (DIFERENTES de produccion!) ---
DB_USER=blitz_dev_user
DB_PASSWORD=dev_password_seguro
DB_NAME=blitz_development
```

```
# --- Django ---
SECRET_KEY=otra-secret-key-para-staging

# --- Firebase ---
FIREBASE_CREDENTIALS_PATH=squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json

# --- Stripe (TEST keys!) ---
STRIPE_SECRET_KEY=sk_test_xxx
STRIPE_PUBLISHABLE_KEY=pk_test_xxx
STRIPE_WEBHOOK_SECRET=whsec_test_xxx

# --- PGAdmin ---
PGADMIN_EMAIL=admin@blitz.local
PGADMIN_PASSWORD=admin
```

Guardar: `Ctrl+O`, `Enter`, `Ctrl+X`

6.9 Verificar que todo esta en su lugar

```
echo "=== PRODUCCION ==="
ls -la /root/app/infra/
# compose.yaml  .env  nginx/

echo "=== PRODUCCION NGINX ==="
ls -la /root/app/infra/nginx/
# default.conf

echo "=== PRODUCCION FIREBASE ==="
ls -la /root/app/backend/squapup*.json
# squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json

echo "=== STAGING ==="
ls -la /root/app-dev/infra/
# compose.yaml  .env  nginx/

echo "=== STAGING NGINX ==="
ls -la /root/app-dev/infra/nginx/
# default.conf

echo "=== STAGING FIREBASE ==="
ls -la /root/app-dev/backend/squapup*.json
# squapup-3ab7a-firebase-adminsdk-fbsvc-53c50f45a0.json

echo "=== VERIFICAR .env ==="
head -3 /root/app/infra/.env
# COMPOSE_PROJECT_NAME=blitz-prod
head -3 /root/app-dev/infra/.env
# COMPOSE_PROJECT_NAME=blitz-dev
```

---

7. PASO 4: Primer deploy manual

7.1 Actualizar PRODUCCION

> El compose.yaml nuevo tiene el volume mount de Firebase que faltaba.
>
> **IMPORTANTE**: Si tu produccion actual NO tenia `COMPOSE_PROJECT_NAME` en el `.env`, Docker usaba el nombre del directorio (`infra`) como prefijo de contenedores. Al agregar `COMPOSE_PROJECT_NAME=blitz-prod`, Docker crearia contenedores NUEVOS (`blitz-prod-backend-1`) pero los viejos (`infra-backend-1`) seguirian corriendo, causando conflicto de puertos. Por eso hay que bajar los contenedores ANTES de cambiar el `.env`.

```
cd /root/app/infra

# 1. PRIMERO: Bajar los contenedores actuales (con la config vieja)
docker compose down

# 2. AHORA editar el .env (si aun no lo hiciste en paso 6.7)
#    nano .env  ← agregar COMPOSE_PROJECT_NAME=blitz-prod, etc.

# 3. Tambien reemplazar compose.yaml si aun no lo hiciste (paso 6.5)

# 4. Verificar que el compose lee las variables bien
docker compose config | head -5
# Debe mostrar: name: blitz-prod

# 5. Levantar con la nueva configuracion
docker compose up -d

# 6. Verificar que Firebase carga
docker compose exec backend python3 manage.py shell -c "from
api.Authentication.authentication import FirebaseAuthentication; import
firebase_admin; print('Firebase OK:', firebase_admin.get_app().name)"
# Debe mostrar: Firebase OK: [DEFAULT]
```

> **Nota**: El `docker compose down` detiene los contenedores pero NO borra la base de datos (los volumenes se preservan). Tus datos de produccion estan seguros.

7.2 Levantar STAGING

```
cd /root/app-dev/infra

# Verificar configuracion
docker compose config | head -5
# Debe mostrar: name: blitz-dev

# Levantar todo
docker compose up -d
```

```
# Ver estado
docker compose ps
```

Deberias ver algo como:

```
NAME                    STATUS
blitz-dev-backend-1     Up (healthy)
blitz-dev-frontend-1    Up
blitz-dev-db-1          Up (healthy)
blitz-dev-redis-1       Up (healthy)
blitz-dev-nginx-1       Up
```

> Nota: Si la imagen `:dev` no existe todavia en GHCR, primero haz un push a `development` (ver Paso 5).

7.3 Verificar que no chocan

```
# Ver TODOS los contenedores de ambos entornos
docker ps --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"
```

Debes ver dos grupos separados:

```
blitz-prod-nginx-1       0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp    Up
blitz-prod-backend-1     0.0.0.0:8000->8000/tcp                       Up
blitz-dev-nginx-1        0.0.0.0:8080->80/tcp                         Up
blitz-dev-backend-1      0.0.0.0:8001->8000/tcp                       Up
```

7.4 Verificar volumenes separados

```
docker volume ls | grep blitz
```

```
blitz-prod_postgres_data     ← BD de produccion
blitz-dev_postgres_data      ← BD de staging (SEPARADA!)
```

---

8. PASO 5: Probar el flujo automatico

8.1 Deploy a Staging

En tu computadora local:

```
```

```
cd backend
git checkout development
echo "# test staging" >> README.md
git add README.md
git commit -m "test: verificar deploy automatico a staging"
git push origin development
```

8.2 Ver el workflow en GitHub

1. Ve a tu repo > pestana **Actions**
2. Veras **"Build & Deploy Backend"** ejecutandose
3. Clic para ver los 3 jobs:

```
setup           ☑ (detecta: environment=staging, image_tag=dev)
build-and-push  🔄 (construyendo imagen...)
deploy          ⏳ (esperando build...)
```

8.3 Verificar en el VPS

```
ssh root@85.215.xxx.xxx
cd /root/app-dev/infra
docker compose ps
docker compose logs backend --tail 20
```

8.4 Deploy a Produccion

```
cd backend
git checkout main
git merge development
git push origin main
```

Si activaste "Required reviewers", aprueba en Actions > Review deployments.

---

9. Como acceder a cada entorno

Produccion

Servicio	URL
Frontend	`https://jesuslab135.com`
Backend API	`https://jesuslab135.com/api/`

Servicio	URL
WebSocket	`wss://jesuslab135.com/ws/`
Swagger	`https://jesuslab135.com/api/schema/swagger-ui/`

Staging

Servicio	URL
Frontend	`http://jesuslab135.com:8080`
Backend API	`http://jesuslab135.com:8080/api/`
WebSocket	`ws://jesuslab135.com:8080/ws/`
Swagger	`http://jesuslab135.com:8080/api/schema/swagger-ui/`

PGAdmin (solo si se activa)

```
# Activar en staging
cd /root/app-dev/infra
docker compose --profile tools up -d pgadmin
# → http://jesuslab135.com:6060
```

---

10. Diagrama del flujo completo

```
DEVELOPER (tu computadora)
│
├── git push development ──────────────────────────────┐
│                                                      │
├── git push main ──────────────────────────┐          │
│                                           │          │
│                                           ▼          ▼
│                                     ┌── GITHUB ACTIONS ──┐
│                                     │                    │
│                                     │  1. setup          │
│                                     │     main → prod    │
│                                     │     dev  → staging │
│                                     │                    │
│                                     │  2. build-and-push │
│                                     │     main → :latest │
│                                     │     dev  → :dev    │
│                                     │                    │
│                                     │  3. deploy (SSH)   │
│                                     └────────┬───────────┘
│                                              │
│                                   ┌──────────┴──────────┐
│                                   │                     │
```

```
│                          ┌──────▼──────┐          ┌──────▼──────┐
│                          │ /root/app/  │          │/root/app-dev│
│                          │   infra/    │          │   infra/    │
│                          │             │          │             │
│                          │ blitz-prod  │          │ blitz-dev   │
│                          │ :latest     │          │ :dev        │
│                          │ port 80/443 │          │ port 8080   │
│                          └─────────────┘          └─────────────┘
│                                │                        │
│                                │                        │
│                                ▼                        ▼
│                        ┌────────────────┐        ┌────────────────┐
│                        │ PRODUCCION     │        │ STAGING        │
│                        │                │        │                │
│                        │ nginx :80      │        │ nginx :8080    │
│                        │ backend:8000   │        │ backend:8001   │
│                        │ db (separada)  │        │ db (separada)  │
│                        │ redis          │        │ redis          │
│                        │ certbot        │        │ (sin SSL)      │
│                        └────────────────┘        └────────────────┘
│
│  https://jesuslab135.com          http://jesuslab135.com:8080
```

---

11. Troubleshooting

"Error: port is already allocated"

```
# Ver que usa el puerto
ss -tlnp | grep 8080
# Si es un contenedor viejo:
docker ps -a | grep 8080
docker stop <container_id> && docker rm <container_id>
# Re-levantar
cd /root/app-dev/infra && docker compose up -d
```

"manifest unknown" o "image not found"

La imagen `:dev` no existe todavia. Haz push a `development` primero, o ejecuta el workflow manualmente:

1. GitHub > repo > Actions
2. "Build & Deploy Backend" > "Run workflow"
3. Selecciona rama `development`
4. "Run workflow"

"Permission denied" en deploy SSH

```
# Verificar conexion desde tu computadora
ssh root@85.215.xxx.xxx "echo OK"
```

```
# Si falla: regenerar clave y actualizar VPS_SSH_KEY en GitHub
```

Firebase no inicializa

```
cd /root/app/infra  # (o app-dev/infra para staging)

# 1. Verificar que el JSON existe en el host
ls -la ../backend/squapup*.json

# 2. Verificar que el volume mount funciona
docker inspect $(docker compose ps -q backend) | grep -A 5 "Mounts"

# 3. Verificar que el archivo llego al contenedor
docker compose exec backend ls -la "/app/core/squapup-3ab7a-firebase-adminsdk-
fbsvc-53c50f45a0.json"

# 4. Verificar que Firebase carga via Django
docker compose exec backend python3 manage.py shell -c "from
api.Authentication.authentication import FirebaseAuthentication; import
firebase_admin; print('Firebase OK:', firebase_admin.get_app().name)"
```

Los contenedores arrancan pero la app no funciona

```
docker compose logs backend --tail 50
# Errores comunes:
# "could not connect to server" → esperar, db no esta lista aun
# "FIREBASE_CREDENTIALS" → verificar volume mount (ver arriba)
# "No module named" → imagen desactualizada, hacer pull
```

"db is unhealthy"

```
docker compose logs db --tail 30
# Si fallo por credenciales:
docker compose down
docker volume rm blitz-dev_postgres_data  # CUIDADO: borra datos de staging
docker compose up -d
```

Nginx no arranca ("ssl_certificate not found")

Solo pasa en produccion si los certificados no existen:

```
# Opcion 1: Ejecutar init-letsencrypt.sh (primera vez)
# Opcion 2: Temporalmente usar config sin SSL
cp /root/app-dev/infra/nginx/default.conf /root/app/infra/nginx/default.conf
```

```
docker compose restart nginx
# Luego configurar SSL
```

---

12. Referencia de comandos

Produccion

```
cd /root/app/infra

docker compose ps                          # Estado de los contenedores
docker compose logs backend --tail 50      # Logs del backend
docker compose logs -f                     # Logs en tiempo real (Ctrl+C para
salir)
docker compose restart backend             # Reiniciar un servicio
docker compose down && docker compose up -d  # Reiniciar todo
docker compose pull backend                # Descargar imagen nueva
docker compose exec backend python3 manage.py shell   # Django shell
docker compose exec backend python3 manage.py migrate  # Migraciones
docker compose config                      # Ver configuracion resuelta
```

Staging

```
cd /root/app-dev/infra
# Mismos comandos que produccion, pero desde este directorio
```

Comandos globales

```
# Ver TODOS los contenedores
docker ps --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"

# Ver volumenes
docker volume ls | grep blitz

# Uso de disco
docker system df

# Limpiar imagenes viejas
docker image prune -f

# Activar/desactivar PGAdmin
cd /root/app-dev/infra
docker compose --profile tools up -d pgadmin    # activar
docker compose --profile tools stop pgadmin     # desactivar
```

---

13. Checklist final

GitHub (hacer en AMBOS repos: backend y frontend)

- ☐ Secret de repositorio: `VPS_HOST` (IP del VPS)
- ☐ Secret de repositorio: `VPS_USER` (normalmente `root`)
- ☐ Secret de repositorio: `VPS_SSH_KEY` (clave privada SSH completa)
- ☐ Environment `production` creado con `VPS_DEPLOY_PATH` = `/root/app/infra`
- ☐ Environment `staging` creado con `VPS_DEPLOY_PATH` = `/root/app-dev/infra`
- ☐ NO existe `VPS_DEPLOY_PATH` a nivel de repositorio (solo en environments)

Archivos en el repositorio (commitear y pushear)

- ☐ `backend/Dockerfile` — rutas estandarizadas a `/app`
- ☐ `backend/scripts/entrypoint.sh` — ruta estandarizada a `/app/core`
- ☐ `frontend/Dockerfile` — rutas estandarizadas a `/app`
- ☐ `infra/compose.yaml` — parametrizado + volume mount Firebase
- ☐ `backend/.github/workflows/deploy.yml` — multi-entorno
- ☐ `frontend/.github/workflows/deploy.yml` — multi-entorno

VPS — Produccion (/root/app/)

- ☐ `/root/app/infra/compose.yaml` (version nueva con volume mount)
- ☐ `/root/app/infra/.env` con `COMPOSE_PROJECT_NAME=blitz-prod`
- ☐ `/root/app/infra/.env` tiene `COMPOSE_PROFILES=ssl`
- ☐ `/root/app/infra/.env` tiene `IMAGE_TAG=latest`
- ☐ `/root/app/infra/.env` tiene `BACKEND_PORT=8000`
- ☐ `/root/app/infra/.env` tiene `HTTP_PORT=80` y `HTTPS_PORT=443`
- ☐ `/root/app/infra/nginx/default.conf` (con SSL)
- ☐ `/root/app/backend/squapup-...json` (Firebase credentials)
- ☐ Firebase verificado: `Firebase OK: [DEFAULT]`

VPS — Staging (/root/app-dev/)

- ☐ `/root/app-dev/infra/compose.yaml` (mismo archivo que produccion)
- ☐ `/root/app-dev/infra/.env` con `COMPOSE_PROJECT_NAME=blitz-dev`
- ☐ `/root/app-dev/infra/.env` NO tiene `COMPOSE_PROFILES=ssl`
- ☐ `/root/app-dev/infra/.env` tiene `IMAGE_TAG=dev`
- ☐ `/root/app-dev/infra/.env` tiene `BACKEND_PORT=8001`
- ☐ `/root/app-dev/infra/.env` tiene `HTTP_PORT=8080`
- ☐ `/root/app-dev/infra/.env` tiene credenciales de BD DIFERENTES
- ☐ `/root/app-dev/infra/.env` tiene Stripe TEST keys
- ☐ `/root/app-dev/infra/nginx/default.conf` (sin SSL, copiado de staging.conf)
- ☐ `/root/app-dev/backend/squapup-...json` (copia del Firebase JSON)

Verificacion final

- ☐ `docker compose ps` en produccion muestra prefijo `blitz-prod-`
- ☐ `docker compose ps` en staging muestra prefijo `blitz-dev-`

- [ ] `docker volume ls | grep blitz` muestra volumenes separados
- [ ] `https://jesuslab135.com` carga (produccion)
- [ ] `http://jesuslab135.com:8080` carga (staging)
- [ ] Push a `development` triggerea deploy a staging
- [ ] Push a `main` triggerea deploy a produccion

Apendice: Tabla de puertos

Puerto	Servicio	Entorno
80	Nginx HTTP	Produccion (redirige a 443)
443	Nginx HTTPS	Produccion (SSL)
8000	Backend	Produccion
5050	PGAdmin	Produccion (si se activa)
8080	Nginx HTTP	Staging
8001	Backend	Staging
6060	PGAdmin	Staging (si se activa)

Puertos internos (no expuestos)

Puerto	Servicio
5432	PostgreSQL (solo dentro de la red Docker)
6379	Redis (solo dentro de la red Docker)

Apendice: Estructura final del VPS

```
/root/
├── app/                              ← PRODUCCION
│   ├── backend/
│   │   └── squapup-...json           ← Firebase credentials (manual)
│   ├── frontend/                     ← (para futuros secrets)
│   └── infra/
│       ├── compose.yaml              ← Archivo parametrizado
│       ├── .env                      ← blitz-prod, :latest, port 80
│       └── nginx/
│           └── default.conf          ← Con SSL (Let's Encrypt)
│
└── app-dev/                          ← STAGING
    ├── backend/
    │   └── squapup-...json           ← Copia del Firebase JSON
    ├── frontend/                     ← (para futuros secrets)
    └── infra/
        ├── compose.yaml              ← Mismo archivo parametrizado
```

```
        ├── .env                      ← blitz-dev, :dev, port 8080
        └── nginx/
            └── default.conf          ← Sin SSL (HTTP simple)
```

```
Contenedores Docker en el servidor:

Produccion (blitz-prod):              Staging (blitz-dev):
├── blitz-prod-nginx-1                ├── blitz-dev-nginx-1
├── blitz-prod-backend-1              ├── blitz-dev-backend-1
├── blitz-prod-frontend-1             ├── blitz-dev-frontend-1
├── blitz-prod-db-1                   ├── blitz-dev-db-1
├── blitz-prod-redis-1                ├── blitz-dev-redis-1
└── blitz-prod-certbot-1              └── (sin certbot)

Volumenes (datos separados):
├── blitz-prod_postgres_data          ├── blitz-dev_postgres_data
├── blitz-prod_redis_data             ├── blitz-dev_redis_data
└── blitz-prod_frontend_dist          └── blitz-dev_frontend_dist

Ruta INTERNA del contenedor (igual en ambos):
└── /app/core/                        └── /app/core/
    ├── manage.py                         ├── manage.py
    └── squapup-...json (mount)           └── squapup-...json (mount)
```